Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Inverted Index and Code-Based Search Engine

Team Name: *Team #*	*Code Acer* *DSCPP-III-2026-T112*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Gupta, Anmol – 2510385219* *anmolgupta1523@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Kumar, Harshit – 2510012451* *harshitkumarlearning@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Goyal, Atharva – 2510360047* *atharvagoyal45@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Nainwal, Navneet – 2510010627* *nainwalnavneet@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract

The project wants to build a fast code search engine. It uses an index to help find keyword quickly across many source-code files. The system will go through the files pull out pieces like words & symbols and make an index. This index will connect each keyword to the files and locations where it appears. To make things faster the system will uses DSA (Data Structures and Algorithms) like hashing, linked lists, trees, sorting, searching and priority queues. It will search for thing like keyword, functions, variables and classes in programming languages such as C++, Java and Python and also search in PDF, Word Files. The results will be ranked by how relevant they are, to the search. This project shows how DSA (Data Structures and Algorithms) can be used in life to build a working search system.

Updated Project Approach and Architecture

The system follows a step-by-step approach to index and search source-code repositories. First the Directory Walker reads supported source files using C++ file streams and assigns each file a document ID. Next the Lexical Tokenizer removes comments and other noise turning the source code into tokens.

These tokens live in an Inverted Index that holds an posting lists. The posting lists link each keyword to the documents where it appears and to the locations within those documents. Hashing makes index access quick and linked lists and other data-structure concepts keep references and search data organized.

When a user types a query the Query Engine splits the query into tokens with a lexer. Applies Boolean logic. Of scanning every file the Query Engine looks up the relevant posting lists in the Inverted Index. The documents that match are sent to a Min-Heap Ranker, which scores each result with TF-IDF to determine relevance. Then orders the results.

The output shows the user the relevant source files, their document IDs and the exact code locations. The web page is created using the HTML, CSS and JS and the data structure and core logic is made through the C/C++. The whole project is a stand-alone search engine, for local repositories.

Tasks Completed

Task Completed	Team Member
Ingestion Pipeline: Recursively scans source-code files, assigns unique document IDs, and prepares files for indexing by extracting and normalizing text.	Anmol Gupta
Index Insertion Logic: Stores each unique word in a hash table and maintains posting lists that record the documents containing the word and its term frequency.	Navneet Nainwal
Query Parsing: Processes user search queries, identifies keywords and logical operators, and retrieves matching documents using the inverted index.	Harshit Kumar
Relevance Ranking, TF-IDF and Heap: Calculates document relevance using TF-IDF scores and uses a Min-Heap to efficiently select the top-K most relevant search results.	Atharva Goyal

Challenges/Roadblocks

The main Challenges faced when we build the Inverted Index project are how to manage a number of documents without slowing down and how to design a good data structure that keeps words and the documents in which each word appears. Another Challenge is to process the text by taking care of punctuation marks removing duplicate words handling different letter cases and dropping words that add no value.

Searching efficiently is also a Challenge especially when we type in keywords. The project must return documents fast without looking through every single document each time. Memory management and keeping the index correct while adding or removing documents are Challenges.

To overcome these challenges we plan to use data structures like hash tables /maps and linked lists to store and fetch index information very fast.
We have tested the system by varying it through the multiple numbers and sizes of the document to find the problems and converts the lowercase deleting the punctuation mark and the breaking the text into token by using the pre processing methods.

We will make the inverted index search engine faster and efficient and memory efficient too by testing and debugging the code and also checks that it should correctly handles all the situations easily.

Tasks Pending

Task Pending	Team Member (to complete the task)
Lexical Tokenizer and Testing & Documentation: Fetch the tokens from the source files by removing comment & string literal and then tests the systems functionality & document the implementation.	Anmol Gupta
Hash Table Implementation and Memory Management: Implement a Hash table using the Hashing algorithm for efficient word lookup. Also manages dynamic memory allocation i.e DMA and deallocation to prevent memory leak.	Navneet Nainwal
Booelan Operations and Text Preprocessing: Implements boolean operation & pre-processes text by normalizing word & removing unnecessary character for accurate searching.	Harshit Kumar
CLI Interface and Module Integration: Develops the Command Line Interface (CLI) for accepting folder paths & search queries, integrates all modules & displays ranked result with execution times.	Atharva Goyal

Project Outcome/Deliverables

The project is currently in the development stage. The project requirement, system design, architecture & overall approach have been defined. The major components such as Directory Walker, Lexical Tokenizer, Inverted Index, Query Engine & Ranking Module have been planned.

The initial design & selection of DSA techniques such as Hashing, Linked Lists, Searching, Sorting and Priority Queue have also been completed. The next major step are implementing source code processing building the Inverted Index developing the search & ranking modules & creating the user interface.

Progress Overview

This project will provide a code-based search engine that works on local source-code repositories. The main deliverables are:
- A word-based index that has dynamic posting lists to allow quick keyword lookup.
- Not to scan file every time it automatically scans the keyword.
- Handles queries by using the post links operation.
- For making the relevant result come first TD IDF score and a priority ques has been implemented
- Prevents to rebuild the index every time by using binary index.
- Keywords, Functions, Variables and Classes can be finds by using the search engine.
- Hashing, Pointers, Recursion, Searhing, Sorting and Priority Queues efficiently has been used by the search engine.
- For faster, efficient and accurate search engine Source Code, Documentation and Test Cases has been used.

Codebase Information

Repository Link: https://github.com/Anmol-Gupta2007/Inverted-Index-and-Code-Based-Search-Engine.git

Branch: Main

Important Commits and Team Contributions:

***Anmol Gupta* – Project Documentation and Interface:** Created index.html for the project interface and README.md for project documentation and setup instructions.

***Harshit Kumar* – Query Parsing:** Created query_parser.cpp and query_parser.h to support query parsing and search-query processing.

***Atharva Goyal* – Ranking and Result Selection:** Created minheap.h, tfidf.c, and tfidf.h for Min-Heap-based result selection and TFIDF relevance scoring.

***Navneet Nainwal* – Inverted Index Engine:** Created inverted_index.c, inverted_index.h, and test_inverted_index.c for the core inverted index implementation, its interface, and testing.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Keyword Search Test	Pass	Tested searches using single keywords and checked the returned documents.
Multiple Keyword Search	Pass	Testing and improving search results when multiple keywords are provided.
Large Dataset	Fail	Larger number of documents to evaluate performance and memory usage.
Memory Testing	Pass	Check memory allocation and deallocation, especially if you're using C or C++.

Deliverables Progress

1. **Inverted Index Data Structure –** In Progress

Inverted Index maps the words of the document that has been implemeted.

2. **Text Pre-processing –** In Progress

Implementation of Tokenizor , Case Normalization and Handling of punctuation.

3. **Document Indexing –** In Progress

Processed Documents adds to Inverted Index.

4. **Search Functionality –** In Progress

For improving the multiple keyword searches and efficiency keyword based searching is used.

5. **Testing and Validation –** In Progress

Sample testing from the documents has been done and testing on large dataset is about to be done.

6. **Performance Optimization -** Pending

Further optimization of time and memory usage will be carried out after the core functionality is finalized.